

FlowGuard 5000 Pump Health Monitoring

Mathematical Background Notes
Version 2

ENS 184 — Numerical Methods for Engineers

May 1, 2026

Contents

1	Introduction	3
2	Polynomial Least-Squares Regression	3
2.1	The Fitting Problem	3
2.2	The Design Matrix	4
2.3	Derivation of the Normal Equations	4
2.4	Numerical Stability and the Backslash Operator	5
2.5	Horner's Method for Efficient Evaluation	5
3	Exponential Models and Log-Linearisation	6
3.1	Why Exponential Growth?	6
3.2	The Linearisation Technique	6
3.3	Parameter Recovery	7
3.4	Validity Conditions and Error Structure	7
3.5	Extrapolation and Its Risks	7
4	Goodness-of-Fit Metrics	8
4.1	Residuals	8
4.2	Root-Mean-Square Error (RMSE)	8
4.3	Coefficient of Determination (R^2)	8
4.4	Interpreting the Metrics Together	9
4.5	Edge Cases	9
5	Root-Finding: The Bisection Method	10
5.1	The Root-Finding Problem	10
5.2	The Intermediate Value Theorem and Bracket Finding	10
5.3	The Bisection Algorithm	10
5.4	Convergence Analysis	11
5.5	Comparison with Other Root-Finding Methods	11
5.6	Stopping Criteria	12
6	Normalised Health Indicators	12
6.1	Normalisation in Engineering Monitoring	12
6.2	The Health Index	12
6.3	Clamping and Its Justification	13
6.4	Visualisation Principles	13

7	Summary of Methods	13
8	Further Reading	13

1 Introduction

This document is a **mathematical reference** for the numerical methods used in the ENS 184 mini-project. It is not a step-by-step implementation guide — it is the theoretical foundation you should understand before writing any code, and the resource you return to when something does not behave as expected. Each section is self-contained so that you can read them in any order.

The project analyses two physical channels of a centrifugal pump using four main numerical methods:

Channel	Input	Output	Model	Method
Calibration	Voltage V (mV)	Pressure P (bar)	Cubic polynomial	Least-squares regression
Degradation	Hours h (h)	Vibration ξ (mm/s _{rms})	Exponential $a e^{bh}$	Log-linearisation
Evaluation	Measured + fitted	—	—	RMSE and R^2 metrics
Prognosis	Fitted model	Failure hour h^*	—	Bisection root-finding

2 Polynomial Least-Squares Regression

Core Idea

Given n data pairs (x_i, y_i) , find the degree- k polynomial

$$P(x) = c_0 + c_1x + c_2x^2 + \cdots + c_kx^k$$

whose graph passes as close to all n points as possible, in the sense of minimising the total squared prediction error.

2.1 The Fitting Problem

A degree- k polynomial is defined by $k + 1$ coefficients $\mathbf{c} = [c_0, c_1, \dots, c_k]^\top$. When exactly $n = k + 1$ data pairs are available there is a unique polynomial of degree $\leq k$ that passes through every point — the *interpolating* polynomial. In engineering it is common to have far more measurements than coefficients ($n \gg k + 1$). Writing one equation per measurement yields an *overdetermined* system:

$$c_0 + c_1x_i + \cdots + c_kx_i^k = y_i, \quad i = 1, \dots, n.$$

There is generally no $\mathbf{c}$ that satisfies all n equations exactly. Instead we seek $\mathbf{c}$ that minimises the **sum of squared residuals**:

$$S(\mathbf{c}) = \sum_{i=1}^n (y_i - P(x_i))^2 = \|\mathbf{y} - \mathbf{Z}\mathbf{c}\|_2^2,$$

where $\mathbf{y} = [y_1, \dots, y_n]^\top$. This is the **least-squares** problem. By squaring the residuals, large deviations are penalised more heavily than small ones, and the minimiser has a clean closed-form solution.

Geometric Interpretation

In $\mathbb{R}^n$, the vector $\mathbf{y}$ does not generally lie in the column space of $\mathbf{Z}$ when the system is overdetermined. The least-squares solution $\hat{\mathbf{c}}$ produces the fitted vector $\hat{\mathbf{y}} = \mathbf{Z}\hat{\mathbf{c}}$ that is the *orthogonal projection* of $\mathbf{y}$ onto the column space of $\mathbf{Z}$. Orthogonality requires the

residual vector $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ to be perpendicular to every column of $\mathbf{Z}$:

$$\mathbf{Z}^\top (\mathbf{y} - \mathbf{Z}\hat{\mathbf{c}}) = \mathbf{0},$$

which is exactly the Normal Equations derived in Section 2.3.

2.2 The Design Matrix

Collect all n equations into a single matrix equation by defining the **design matrix** $\mathbf{Z} \in \mathbb{R}^{n \times (k+1)}$:

$$\mathbf{Z} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^k \\ 1 & x_2 & x_2^2 & \cdots & x_2^k \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^k \end{bmatrix}$$

Column j of $\mathbf{Z}$ contains x_i^{j-1} for $i = 1, \dots, n$, at $j = 1, 2, \dots, k+1$. With this definition the system of n equations becomes the compact matrix equation $\mathbf{Z}\mathbf{c} \approx \mathbf{y}$, which holds with equality only when $\mathbf{y}$ lies exactly in the column space of $\mathbf{Z}$ (i.e. the exactly-determined or consistent case).

The design matrix is also called the *Vandermonde matrix*. It is square when $n = k+1$ and tall (more rows than columns) when $n > k+1$.

2.3 Derivation of the Normal Equations

To minimise $S(\mathbf{c}) = \|\mathbf{y} - \mathbf{Z}\mathbf{c}\|_2^2$, expand the squared norm:

$$S(\mathbf{c}) = \mathbf{y}^\top \mathbf{y} - 2\mathbf{c}^\top \mathbf{Z}^\top \mathbf{y} + \mathbf{c}^\top (\mathbf{Z}^\top \mathbf{Z}) \mathbf{c}.$$

Take the gradient with respect to $\mathbf{c}$ and set it to zero:

$$\nabla_{\mathbf{c}} S = -2\mathbf{Z}^\top \mathbf{y} + 2(\mathbf{Z}^\top \mathbf{Z})\mathbf{c} = \mathbf{0}.$$

Rearranging gives the **Normal Equations**:

$$(\mathbf{Z}^\top \mathbf{Z})\mathbf{c} = \mathbf{Z}^\top \mathbf{y}.$$

This is a square $(k+1) \times (k+1)$ linear system. When the columns of $\mathbf{Z}$ are linearly independent — which requires that the predictor values x_i are not all identical and $n \geq k+1$ — the Gram matrix $\mathbf{Z}^\top \mathbf{Z}$ is symmetric positive definite and the system has a unique solution.

Example: Quadratic Fit to Three Points

Data: $(0, 1)$, $(1, 3)$, $(2, 9)$. Fit a degree-2 polynomial.

$$\mathbf{Z} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 3 \\ 9 \end{bmatrix}.$$

Here $n = k+1 = 3$, so $\mathbf{Z}$ is square. Solving $(\mathbf{Z}^\top \mathbf{Z})\mathbf{c} = \mathbf{Z}^\top \mathbf{y}$ gives $\mathbf{c} = [1, 0, 2]^\top$, i.e. $P(x) = 1 + 2x^2$.

Check: $P(0) = 1 \checkmark$, $P(1) = 3 \checkmark$, $P(2) = 9 \checkmark$.

When $n = k+1$ the Normal Equations recover the unique interpolating polynomial. When $n > k+1$ the solution minimises the total squared residual.

2.4 Numerical Stability and the Backslash Operator

The algebraically straightforward implementation of least-squares would compute:

$$\hat{\mathbf{c}} = (\mathbf{Z}^\top \mathbf{Z})^{-1} \mathbf{Z}^\top \mathbf{y}$$

by explicitly inverting the Gram matrix. This is **numerically unreliable** in practice.

The **condition number** of a matrix $\mathbf{A}$ quantifies how much small perturbations to $\mathbf{A}$ (such as floating-point roundoff errors) are amplified in the solution of $\mathbf{Ax} = \mathbf{b}$:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|.$$

A large κ means the system is ill-conditioned: the solution changes dramatically for tiny changes in the right-hand side. A key identity for the Gram matrix is:

$$\kappa(\mathbf{Z}^\top \mathbf{Z}) = \kappa(\mathbf{Z})^2.$$

Forming $\mathbf{Z}^\top \mathbf{Z}$ therefore *squares* the conditioning problem. If $\kappa(\mathbf{Z}) = 10^4$ (not unusual for a degree-3 polynomial over a wide voltage range), then $\kappa(\mathbf{Z}^\top \mathbf{Z}) = 10^8$, and floating-point arithmetic at double precision ($\approx 10^{-16}$ relative error) can lose up to 8 significant digits in the solution.

Modern numerical solvers work directly with $\mathbf{Z}$ using a **QR factorisation** (or singular value decomposition), which requires no squaring and preserves the conditioning of the original matrix. In Scilab the backslash operator applied to the overdetermined system $\mathbf{Zc} \approx \mathbf{y}$ automatically uses such a factorisation and returns the least-squares solution without ever forming $\mathbf{Z}^\top \mathbf{Z}$ explicitly.

Never Use `inv()` to Solve Linear Systems

`inv(A)` computes the matrix inverse explicitly and then multiplies. This doubles the number of floating-point operations compared to a direct factorisation-based solver and is unnecessarily sensitive to conditioning. Even for well-conditioned problems it is slower and less accurate. Use the backslash operator to solve $\mathbf{Ax} = \mathbf{b}$ directly.

2.5 Horner's Method for Efficient Evaluation

Once coefficients $\mathbf{c}$ are known, the polynomial must be evaluated at many query points. The *direct* approach computes $x^0, x^1, \dots, x^k$ separately and sums the terms, requiring $k(k+1)/2$ multiplications — $O(k^2)$ work.

Horner's method avoids redundant multiplications by exploiting a nested factorisation. Starting from the standard form:

$$P(x) = c_0 + c_1x + c_2x^2 + \dots + c_kx^k,$$

factor out x from the last k terms, then from the last $k-1$ terms, and so on:

$$P(x) = c_0 + x(c_1 + x(c_2 + \dots + x(c_{k-1} + xc_k) \dots)).$$

This requires exactly k multiplications and k additions — $O(k)$ work — regardless of the degree. The algorithm processes coefficients from *highest* degree to *lowest*:

1. Initialise: $r \leftarrow c_k$
2. For $j = k-1, k-2, \dots, 0$: $r \leftarrow c_j + x \cdot r$
3. Return r

Horner on a Cubic

$P(x) = 2 + 0x - x^2 + 3x^3$, so $\mathbf{c} = [2, 0, -1, 3]^\top$, $k = 3$. Evaluate at $x = 2$:

$$r \leftarrow 3$$

$$r \leftarrow -1 + 2 \cdot 3 = 5$$

$$r \leftarrow 0 + 2 \cdot 5 = 10$$

$$r \leftarrow 2 + 2 \cdot 10 = 22$$

Direct check: $2 + 0 - 4 + 24 = 22$ ✓.

3 Exponential Models and Log-Linearisation

Core Idea

The bearing degradation model $\xi(h) = a e^{bh}$ is nonlinear in its parameters, but taking the natural logarithm of both sides converts it into a linear equation in h . The resulting linear system is solved with the same design matrix and Normal Equations machinery as polynomial regression.

3.1 Why Exponential Growth?

Rolling-element bearing failure is driven by surface fatigue: microscopic cracks form on the rolling surfaces and propagate under cyclic contact loading. As crack density increases, the energy dissipated per revolution — manifested as vibration — grows at an *accelerating* rate. Empirical evidence and damage-mechanics models both support an exponential relationship between vibration amplitude and accumulated operating hours during the late-stage degradation phase:

$$\xi(h) = a e^{bh}, \quad a > 0, \quad b > 0.$$

The parameter a is the vibration level at $h = 0$ (the start of the monitoring period, when wear is minimal). The parameter b controls the rate of growth: a larger b means faster acceleration of degradation.

An exponential model with $b > 0$ is always positive and strictly increasing — physically consistent with a bearing that is degrading but not yet broken.

3.2 The Linearisation Technique

The model $\xi(h) = a e^{bh}$ is *nonlinear in its parameters*: doubling a doubles ξ , but doubling b more than doubles ξ at all non-zero hours. The design-matrix / Normal-Equations machinery applies only to models that are **linear in their parameters**. An algebraic transformation resolves this.

Apply the natural logarithm to both sides of the model:

$$\ln \xi = \ln a + b h.$$

Define the *transformed response* $w = \ln \xi$. Since $\ln a$ and b are constants, the equation

$$w = c_1 + c_2 h, \quad c_1 = \ln a, \quad c_2 = b$$

is linear in the unknown parameters c_1 and c_2 , and linear in the predictor h . This is exactly the structure of a degree-1 polynomial in h , so the design matrix approach from Section 2

applies directly, using the two-column design matrix $\mathbf{Z} = [\mathbf{1}, \mathbf{h}] \in \mathbb{R}^{n \times 2}$ and the transformed response vector $\mathbf{w} = \ln \boldsymbol{\xi}$ (natural logarithm applied element-wise). The Normal Equations $(\mathbf{Z}^\top \mathbf{Z})\mathbf{c} = \mathbf{Z}^\top \mathbf{w}$ yield $\hat{\mathbf{c}} = [\hat{c}_1, \hat{c}_2]^\top$.

3.3 Parameter Recovery

The least-squares fit operates in the log-transformed space. The physical parameters of the original exponential model are recovered by inverting the transformation:

$$\hat{a} = e^{\hat{c}_1}, \quad \hat{b} = \hat{c}_2.$$

The growth rate $\hat{b}$ appears directly as the slope coefficient in the linear model and requires no further transformation. The scale factor $\hat{a}$ requires the inverse of the natural logarithm (the exponential function).

3.4 Validity Conditions and Error Structure

The log-transform requires $\xi_i > 0$ for all data points: the logarithm is undefined for zero or negative arguments. A pump in service always produces some vibration, so this condition is physically reasonable.

A subtler point: applying a transformation to the *response* changes the statistical error structure of the fitting problem. The Normal Equations in the log-space minimise

$$S_w = \sum_{i=1}^n (\ln \xi_i - \hat{c}_1 - \hat{c}_2 h_i)^2,$$

not the natural objective in the original scale:

$$S_\xi = \sum_{i=1}^n (\xi_i - \hat{a} e^{\hat{b} h_i})^2.$$

These two criteria give slightly different optimal parameters because the log-transform implicitly assigns more weight to small vibration values. In practice, when the exponential model fits well ($R^2 > 0.95$ in the log-space), the two solutions are very close and the distinction is of academic interest only. Log-linearisation is the standard engineering approach because it is fast, deterministic, and requires no iterative solver.

3.5 Extrapolation and Its Risks

Caution on Exponential Extrapolation

An exponential function grows without bound. A model fitted on a finite training range produces predictions for *any* hour value, but the further those values lie outside the training range, the less reliable the prediction. The failure-hour estimate h^* is the model's best quantitative guess; it should always be accompanied by the caveat that it relies on the degradation trend observed so far continuing at the same rate — which may not hold as failure approaches.

4 Goodness-of-Fit Metrics

Core Idea

Two complementary metrics — RMSE and R^2 — characterise how closely a fitted model tracks the data. RMSE quantifies typical prediction error in physical units; R^2 expresses the fraction of data variability explained by the model on a dimensionless scale. Together they give a complete picture of model quality.

4.1 Residuals

The **residual** at data point i is the signed difference between the measured value and the model's prediction:

$$e_i = y_i - \hat{y}_i.$$

A positive residual means the model under-predicted; negative means it over-predicted. A well-fitted model has residuals that:

- are centred near zero (no systematic bias),
- show no trend when plotted against x_i or $\hat{y}_i$ (random scatter),
- have roughly constant spread (homoscedasticity).

Patterns in the residuals signal that the model is misspecified: the polynomial degree may be too low, the functional form wrong, or the linearisation assumption violated. A quick residual plot is one of the most informative diagnostics available after fitting.

4.2 Root-Mean-Square Error (RMSE)

RMSE is the square root of the mean squared residual:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2}.$$

Key properties:

- RMSE has the *same physical units* as the response variable y . An RMSE of 0.3 bar means the model's typical prediction error is 0.3 bar.
- $\text{RMSE} \geq 0$; a value of 0 means the model predicts every data point exactly.
- Squaring the residuals makes RMSE sensitive to *outliers*: a single large residual can dominate the sum. If large residuals are physically impossible (sensor limits), check for parsing errors first.
- RMSE values from different datasets are comparable only when the response variable has the same units and similar scale.

4.3 Coefficient of Determination (R^2)

R^2 compares the fitted model against the simplest possible baseline — a model that always predicts the sample mean $\bar{y}$:

$$SS_{\text{res}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad SS_{\text{tot}} = \sum_{i=1}^n (y_i - \bar{y})^2,$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}.$$

SS_{tot} measures the total variance in the data (scaled by n); SS_{res} is the unexplained variance that remains after fitting. Their ratio $SS_{\text{res}}/SS_{\text{tot}}$ is the fraction of variance *not* explained by the model, so $1 - SS_{\text{res}}/SS_{\text{tot}}$ is the fraction that *is* explained.

Key properties:

- $R^2 \leq 1$ always. $R^2 = 1$ iff $SS_{\text{res}} = 0$ (perfect fit).
- $R^2 = 0$ when the model performs no better than the mean predictor ($SS_{\text{res}} = SS_{\text{tot}}$).
- $R^2 < 0$ when the model fits *worse* than the constant mean predictor. This signals a seriously misspecified model.
- Unlike RMSE, R^2 is dimensionless and scale-free, making it useful for comparing fits across different physical quantities.

R^2 Can Be Negative and Is Not a Percentage

R^2 is not bounded below by zero. A negative R^2 indicates that the model is worse than the constant predictor — a clear sign that something has gone wrong in the fitting procedure (wrong model family, sign error in the linearisation, etc.). Do not interpret $R^2 = 0.97$ as “97% accurate”; it means the model explains 97% of the variance in the data, which is a different statement.

4.4 Interpreting the Metrics Together

Situation	RMSE	R^2
Perfect fit	0	1
Good engineering fit	small (in context)	close to 1
Model no better than mean	equals std dev of y	0
Worse than mean	larger than std dev of y	negative

The two metrics complement each other. A very high R^2 (e.g. 0.99) with a large RMSE (e.g. 2 bar) means the model captures the trend well but predictions are imprecise in absolute terms. Conversely, a low RMSE does not guarantee a high R^2 if the response has very low total variance. For this project both the polynomial and exponential models should achieve $R^2 > 0.95$ on the provided dataset when implemented correctly.

4.5 Edge Cases

- $SS_{\text{tot}} = 0$: occurs when all n measured values are identical ($y_i = \bar{y}$ for all i), so the denominator is zero. Any model that predicts $\bar{y}$ is as accurate as possible; the convention is to return $R^2 = 1$. Your implementation must guard against division by zero.
- $SS_{\text{res}} = 0$: perfect prediction; $R^2 = 1$ and $\text{RMSE} = 0$ follow directly from the formulas, with no special handling required.
- **Single data point** ($n = 1$): $SS_{\text{tot}} = 0$ always; the edge case above applies.

Quick Verification Tests

- `goodness_of_fit(y, y)` for any non-trivial $\mathbf{y}$: must return $\text{RMSE} = 0$ and $R^2 = 1$.
- With $\hat{\mathbf{y}} = \bar{y} \mathbf{1}$ (constant vector equal to the mean of $\mathbf{y}$): must return $R^2 = 0$ exactly.
- With $\hat{\mathbf{y}} = \mathbf{0}$ (all zeros) and non-zero data: R^2 should be negative.

5 Root-Finding: The Bisection Method

Core Idea

Given a continuous function g and an interval where g changes sign, the bisection method repeatedly halves the interval by evaluating g at the midpoint. It is guaranteed to converge to a root, and the number of steps required to achieve any target precision is predictable in advance.

5.1 The Root-Finding Problem

The degradation model predicts vibration as $\hat{\xi}(h) = \hat{a} e^{\hat{b}h}$. The *failure hour* h^* is the operational hour at which the predicted vibration first reaches the failure threshold ξ_{th} :

$$\hat{\xi}(h^*) = \xi_{\text{th}}.$$

This is equivalent to finding the root of the function

$$g(h) = \hat{\xi}(h) - \xi_{\text{th}} = 0.$$

Since $\hat{\xi}$ is monotone increasing (for $\hat{b} > 0$), there is at most one such h^* . If the fitted vibration never reaches ξ_{th} within the available data range, the function should return $+\infty$ to indicate that no in-range failure is predicted.

5.2 The Intermediate Value Theorem and Bracket Finding

Intermediate Value Theorem (IVT)

If g is continuous on the closed interval $[h_L, h_R]$ and $g(h_L) \cdot g(h_R) < 0$ (i.e. g has opposite signs at the endpoints), then there exists at least one $h^* \in (h_L, h_R)$ such that $g(h^*) = 0$.

An interval satisfying the sign-change condition is called a **bracket**. The existence of a valid bracket is the *only* prerequisite for bisection; no other information about g is required.

In this project, the fitted vibration model is evaluated at the same discrete hour values as the data. A bracket is identified by scanning consecutive pairs (h_i, h_{i+1}) and testing whether the sign of $\hat{\xi}_i - \xi_{\text{th}}$ differs from the sign of $\hat{\xi}_{i+1} - \xi_{\text{th}}$. The first such consecutive pair forms the initial bracket.

Within the bracket, g is not known analytically — only two point values are available at the bracket endpoints. The value of g at any interior point h_M is estimated by **linear interpolation** between the two fitted vibration values:

$$g(h_M) \approx \hat{\xi}(h_L) + \frac{h_M - h_L}{h_R - h_L} (\hat{\xi}(h_R) - \hat{\xi}(h_L)) - \xi_{\text{th}}.$$

This piecewise-linear approximation is justified because the bracket is narrow enough (typically tens of hours) that the exponential is nearly linear within it.

5.3 The Bisection Algorithm

1. **Find bracket:** scan the fitted vibration vector to find adjacent indices i and $i + 1$ where $g(h_i) \cdot g(h_{i+1}) \leq 0$. Set $h_L \leftarrow h_i$, $h_R \leftarrow h_{i+1}$. If no such pair exists, return $h^* = +\infty$.
2. **Bisect:** repeat until $h_R - h_L < \varepsilon$:
 - (a) Compute midpoint: $h_M \leftarrow (h_L + h_R)/2$.

- (b) Evaluate $g(h_M)$ by linear interpolation within the bracket.
 - (c) If $g(h_L) \cdot g(h_M) < 0$: set $h_R \leftarrow h_M$ (root is in the left half).
 - (d) Otherwise: set $h_L \leftarrow h_M$ (root is in the right half or at h_M).
3. Return $h^* \leftarrow (h_L + h_R)/2$.

5.4 Convergence Analysis

Let $\delta_0 = h_R - h_L$ denote the initial bracket width. After each bisection step the bracket width is exactly halved. After n steps:

$$\delta_n = \frac{\delta_0}{2^n}.$$

To guarantee a bracket width smaller than tolerance ε , the required number of steps is:

$$n \geq \left\lceil \log_2 \left(\frac{\delta_0}{\varepsilon} \right) \right\rceil.$$

This formula is one of bisection’s most attractive features: the number of iterations needed to achieve any target precision is known *before* the algorithm runs.

Convergence Example

Initial bracket $\delta_0 = 100$ h; tolerance $\varepsilon = 0.5$ h.

$$n \geq \lceil \log_2(200) \rceil = \lceil 7.64 \rceil = 8 \text{ steps.}$$

Eight iterations suffice regardless of the shape of g within the bracket, as long as the bracket is valid.

Bisection converges **linearly** with convergence rate $\frac{1}{2}$: the error bound is halved at every step. This is expressed by writing the error after step n as $|\varepsilon_n| \leq C \cdot (\frac{1}{2})^n$, where $C = \delta_0$. The “1” in “linear convergence” refers to the fact that $\log |\varepsilon_n|$ decreases linearly with n (one bit of precision gained per step).

5.5 Comparison with Other Root-Finding Methods

Method	Convergence rate	Requires	Failure modes
Bisection	Linear ($\frac{1}{2}$)	Valid bracket	None (guaranteed)
Newton–Raphson	Quadratic	$g'(h)$, good initial guess	Can diverge
Secant	Superlinear (≈ 1.618)	Two initial guesses	Can diverge
False position	Linear (sometimes faster)	Valid bracket	Can stagnate

Newton’s method converges dramatically faster — the number of correct decimal digits roughly doubles at each step — but it requires the derivative $g'(h)$ and a good initial guess; without these it can diverge or cycle. The secant method avoids the derivative by approximating it with a finite difference, but shares Newton’s sensitivity to initial conditions.

For maintenance-critical applications where *robustness* matters more than convergence speed, bisection is the preferred choice. The difference in runtime between 8 and 50 iterations on a modern computer is imperceptible, while a diverging Newton iteration can produce a completely wrong answer without obvious warning.

5.6 Stopping Criteria

Two stopping strategies are common:

- **Bracket-width criterion:** stop when $h_R - h_L < \varepsilon$. This is the *preferred* choice for bisection because the required number of steps is predictable and convergence is guaranteed.
- **Residual criterion:** stop when $|g(h_M)| < \delta$. Appropriate when the absolute magnitude of the function matters, but can be slow if g is nearly flat near the root.

For this project a bracket-width tolerance of $\varepsilon = 0.5$ h (thirty minutes) is more than adequate as a maintenance prediction.

Use a Bounded Loop

Implement bisection with a **for** loop over a finite maximum number of iterations (e.g. 100) rather than an unbounded **while** loop. Even with correct logic, floating-point arithmetic can occasionally prevent the bracket width from reaching exactly zero, causing a **while** loop to run indefinitely. With 100 iterations and a 100 h initial bracket, the bracket width falls below 10^{-28} h — far beyond any physically meaningful precision.

6 Normalised Health Indicators

Core Idea

A **Health Index** maps a physical sensor measurement to a dimensionless score in $[0, 1]$ so that operators can assess equipment condition at a glance, independent of the specific sensor type or failure threshold.

6.1 Normalisation in Engineering Monitoring

Raw sensor measurements — vibration in mm/s_{rms} , pressure in bar, temperature in $^{\circ}\text{C}$ — are only meaningful to engineers who know the corresponding failure limits. **Normalisation** converts each measurement to a common scale by dividing by a reference value (typically the failure threshold or rated capacity):

$$\text{Normalised value} = \frac{\text{Measured value}}{\text{Reference value}}.$$

When the normalised value is 0 the machine is in its best possible state; when it reaches 1 the machine has attained the limit of acceptable operation. This pattern appears across engineering disciplines:

- *State of Charge* in batteries: fraction of nominal capacity.
- *Damage Index* in structural health monitoring: ratio of current stiffness reduction to the failure stiffness reduction.
- *Overall Vibration Severity Bands* in ISO 10816: normalised against machine-class thresholds.

The common thread is that a single number in $[0, 1]$ communicates urgency to a non-specialist without requiring knowledge of physical units or failure criteria.

6.2 The Health Index

For the FlowGuard 5000, the Health Index at operational hour h is:

$$\text{HI}(h) = \frac{\xi(h)}{\xi_{\text{th}}},$$

where $\xi(h)$ is the vibration amplitude (raw or fitted) and ξ_{th} is the manufacturer-specified failure threshold.

HI value	Physical interpretation
0	Zero vibration; pump as-new
$0 < \text{HI} < 1$	Pump operational; bearing degrading
1	Vibration has reached the failure threshold
> 1 (before clamping)	Threshold exceeded; immediate action required

6.3 Clamping and Its Justification

Once $\xi > \xi_{\text{th}}$ the pump should be taken out of service. For dashboard display purposes, an HI greater than 1 conveys no additional actionable information: the go/no-go decision is already made. Values are therefore *clamped* to the interval $[0, 1]$:

$$\text{HI}_{\text{clamped}} = \min(\max(\text{HI}, 0), 1).$$

The lower clamp at 0 handles any data artefacts that might produce a small negative vibration reading. Without clamping, HI values above 1 would distort the vertical axis of the dashboard figure and suggest that a graduated response is possible beyond the threshold, which is operationally misleading.

6.4 Visualisation Principles

The purpose of an engineering dashboard is to answer operational questions unambiguously. For the pump health report, three questions must be answerable from a single glance:

1. How does the measured vibration trend compare with the fitted model?
2. When is the predicted failure, and how much operational time remains?
3. What is the current health status (operational / warning / critical)?

Each question corresponds to specific visual elements in the two-panel figure: data scatter overlaid with the smooth fitted curve; a vertical marker at h^* ; horizontal zone-boundary lines in Panel 2. Consult the project manual (Task D3) for the full element specification.

7 Summary of Methods

Method	Purpose	Key equation	Cost / convergence	Section
Polynomial least-squares	Calibration model	$(\mathbf{Z}^\top \mathbf{Z})\mathbf{c} = \mathbf{Z}^\top \mathbf{y}$	One linear solve	2
Log-linearisation	Degradation model	$\ln \xi = c_1 + c_2 h$	One linear solve	3
RMSE, R^2	Model quality	$R^2 = 1 - SS_{\text{res}}/SS_{\text{tot}}$	$O(n)$ operations	4
Bisection	Failure-hour h^*	Bracket $\rightarrow$ halve $\rightarrow$ repeat	Linear, rate $\frac{1}{2}$	5
Health Index	Dashboard score	$\text{HI} = \xi/\xi_{\text{th}}$, clamped	$O(n)$ operations	6

8 Further Reading

- **Numerical Linear Algebra:** Trefethen & Bau, *Numerical Linear Algebra* (SIAM, 1997) — least-squares, QR decomposition, and conditioning.
- **Matrix Computations:** Golub & Van Loan, *Matrix Computations*, 4th ed. (Johns Hopkins, 2013) — definitive reference on the Normal Equations and stable least-squares solvers.
- **Root-Finding and Numerical Analysis:** Burden, Faires & Burden, *Numerical Analysis*, 10th ed. — bisection, Newton’s method, and convergence theory, Chapter 2.

- **Regression and Model Diagnostics:** Montgomery, Peck & Vining, *Introduction to Linear Regression Analysis* — residual analysis, R^2 , and model selection.
- **Condition Monitoring Standards:** ISO 10816-3:2009 and ISO 20816 — industrial vibration severity criteria and zone classification for rotating machines.
- **Course Lecture Notes:** slides on numerical linear algebra (least-squares, design matrices), root-finding (bisection), and data visualisation.